Week 2: Assignment 2

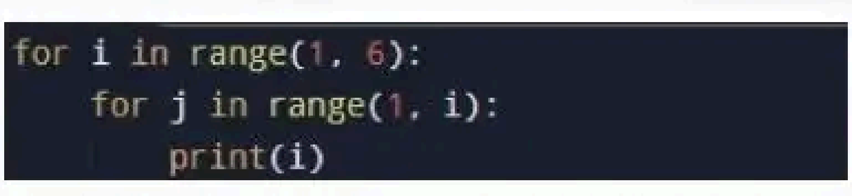
Your last recorded submission was on 2026-01-31, 14:10 IST

Due date: 2026-02-04, 23:59 IST.

Ravi's Nested Loop Implementation

Ravi is asked to print the following pattern using Python:
1
22
333
4444
55555

He knows that a single loop controls the number of lines, so he decides to use **nested for loops**. Ravi writes the following code:



```python
for i in range(1, 6):
    for j in range(1, i):
        print(i)
```

The code runs without errors and prints multiple values on the screen. Ravi assumes that because the inner loop runs multiple times, the digit will automatically appear the required number of times on the same line. However, when the output is observed, the pattern does not match the required format.

The instructor uses Ravi's code as a learning case. Learners are expected to analyze how the **outer loop controls the number of rows**, how the **inner loop controls repetition**, and why the current placement of the print() statement leads to an incorrect output format.

1) How many times does the outer loop execute? *1 point*

○ 4
● 5
○ 6
○ Depends on the inner loop

2) Why does Ravi's code print each number on a new line instead of the same line? *1 point*

● Because print() moves the cursor to the next line by default
○ Because range() creates new lines.
○ Because the inner loop executes first
○ Because nested loops cannot print patterns

3) Ravi's program executes without errors but fails to produce the required pattern. Which of the following best explains the cause? *1 point*

○ The outer loop does not include the final value required for printing the last line
○ The inner loop variable is declared but not explicitly used in the print statement
● The print statement is placed inside the inner loop instead of after it
○ The inner loop executes one fewer iteration than required for each value of i

4) After correcting the inner loop to execute the required number of times, Ravi observes that the digits are still printed on separate lines. Which change is necessary to ensure that each digit appears on the same line before moving to the next line? *1 point*

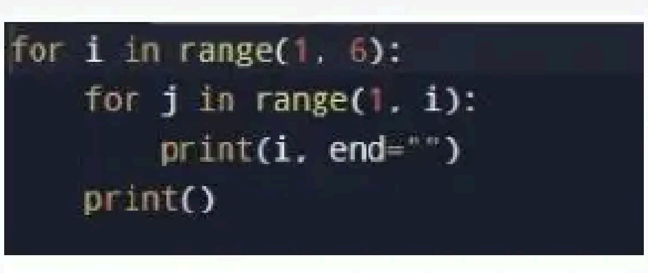
○ Replace print(i) with print() inside the inner loop
○ Move the print(i) statement outside the inner loop
○ Add an empty print() statement at the beginning of the outer loop
● Modify the print statement to suppress the default newline character

5) In the corrected nested-loop program, why is an additional print() statement required immediately after the inner loop completes? *1 point*

● To move the cursor to the next line after printing all digits for the current value of i
○ To reset the value of the loop variable before the next iteration begins
○ To ensure the inner loop terminates before the outer loop continues
○ To prevent the digits printed in different iterations from overlapping

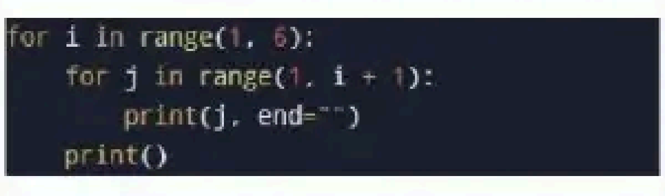
6) Which of the following Python code snippets correctly produces the required output? *1 point*
1
22
333
4444
55555

○



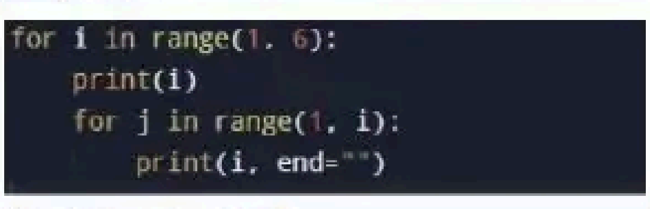
```python
for i in range(1, 6):
    for j in range(1, i):
        print(i, end="")
    print()
```

○



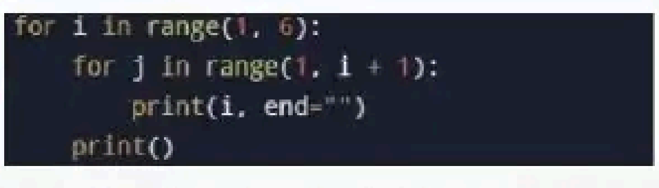
```python
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end="")
    print()
```

○



```python
for i in range(1, 6):
    print(i)
    for j in range(1, i):
        print(i, end="")
```

●



```python
for i in range(1, 6):
    for j in range(1, i + 1):
        print(i, end="")
    print()
```

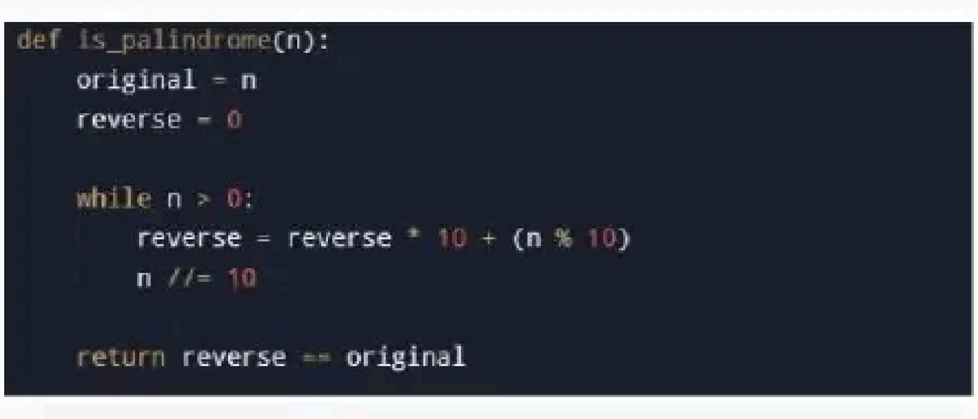
Palindrome Number Checker in Python

Ravi is learning Python and is currently practicing **loops and arithmetic operations.**
As part of his practice, he is given a task to **check whether a given number is a palindrome.**

A **palindrome** number is a number that **reads the same forward and backward.**

Examples:
- 121 → Palindrome
- 456 → Not a Palindrome

Ravi decides **not to convert the number into a string**, because he wants to strengthen his understanding of:
- while loops
- Modulus (%) operator
- Integer division (//)

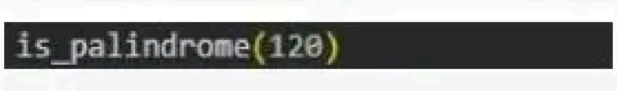


```python
def is_palindrome(n):
    original = n
    reverse = 0

    while n > 0:
        reverse = reverse * 10 + (n % 10)
        n //= 10

    return reverse == original
```

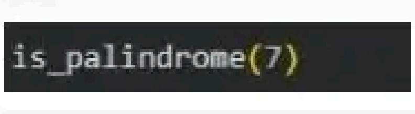
7) What is the final value of reverse when the function is called as: *1 point*



```python
is_palindrome(120)
```

○ 210
○ 120
● 21
○ 12

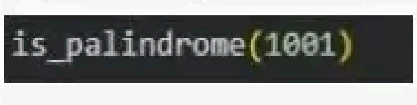
8) What will the function return for the input: *1 point*



```python
is_palindrome(7)
```

○ Error because loop runs only once
● True
○ False
○ None

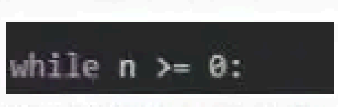
9) How many times does the while loop execute when the function is called as: *1 point*



```python
is_palindrome(1001)
```

○ 2
● 4
○ 3
○ 1

10) Suppose the loop condition is changed to: *1 point*



```python
while n >= 0:
```

What will happen when the function is called with is_palindrome(5)?

● It enters an infinite loop
○ It returns False after one iteration
○ It correctly returns True
○ It raises a runtime error

Smart Discount Calculator

A retail store introduces a Smart Discount Calculator to ensure that discounts are applied in a consistent and predictable manner. Every bill is processed by following a fixed logical sequence. Once a condition is satisfied, the system proceeds forward without revisiting previous checks.

Purchase Amount Based Discount
If the total purchase amount is less than ₹1,000, no discount is applied.
If the total purchase amount is ₹1,000 or more, a 10% purchasebased discount becomes eligible.
Only one purchase-amount condition is checked. After this step, the system moves to customer evaluation.

Customer Type Based Discount
If the customer is a regular customer, no customer-type discount is considered.
If the customer is a premium customer, the system checks whether the purchase amount is at least ₹2,000.
If the customer is premium and the purchase amount is below ₹2,000, the premium discount is ignored.
If the customer is premium and the purchase amount is ₹2,000 or more, a 5% premium discount becomes eligible.

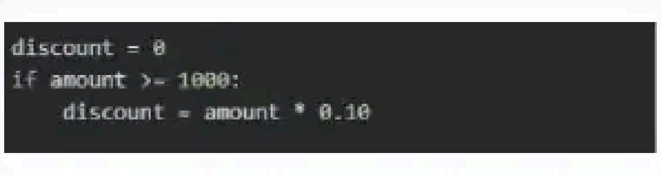
Final Discount Selection Rule
If no discount is eligible from either scheme, the final payable amount remains equal to the purchase amount.
If exactly one discount is eligible, that discount is applied.
If both purchase-based and premium discounts are eligible, the system compares the discount values and applies only the one that results in the higher benefit.
At no point are both discounts applied together.

Safety Rules
If the calculated discount exceeds the purchase amount, the discount is limited to the purchase amount.
If the final payable amount becomes negative, it is reset to zero.

Output Generation
The system displays the original purchase amount, the selected discount, and the final payable amount.
The calculator always produces the same output for the same input, ensuring fairness and transparency in billing.

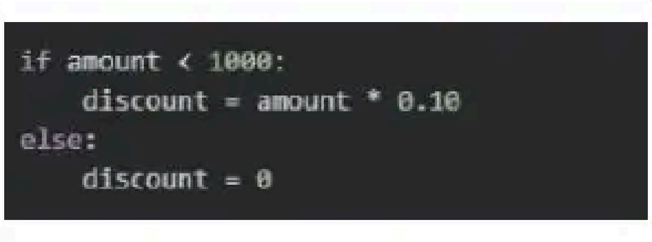
11) A customer is a **regular customer** and makes a purchase of **₹850.** Which code fragment correctly represents the discount logic applied by the calculator? *1 point*

●



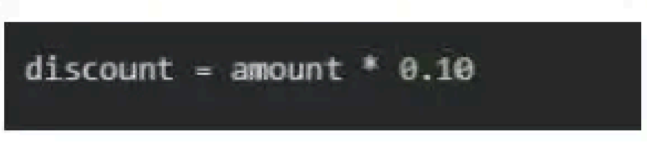
```python
discount = 0
if amount >= 1000:
    discount = amount * 0.10
```

○



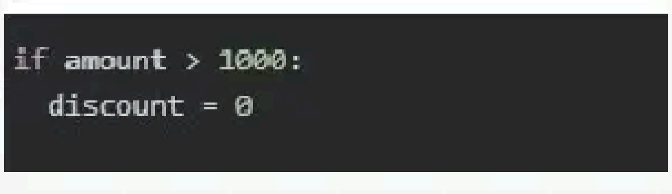
```python
if amount < 1000:
    discount = amount * 0.10
else:
    discount = 0
```

○



```python
discount = amount * 0.10
```

○



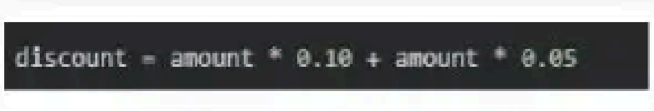
```python
if amount > 1000:
  discount = 0
```

12) A **premium customer** makes a purchase of **₹1,500.** *1 point*
Which discount will the system finally apply?

○ 5% premium discount
○ No discount
● 10% purchase-based discount
○ Both 10% and 5% discounts

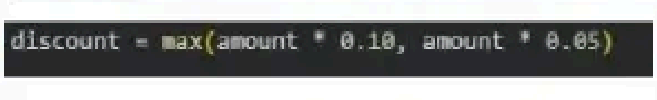
13) A **premium customer** makes a purchase of **₹2,500.** *1 point*
Which logic ensures that only one discount is applied?

○



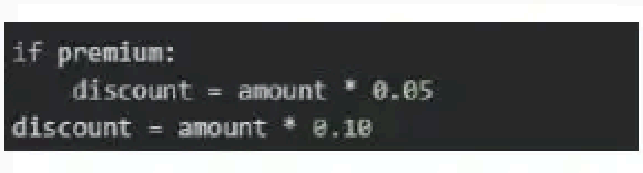
```python
discount = amount * 0.10 + amount * 0.05
```

●



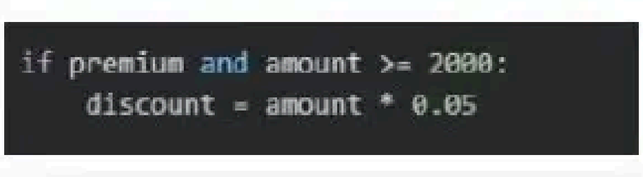
```python
discount = max(amount * 0.10, amount * 0.05)
```

○



```python
if premium:
    discount = amount * 0.05
discount = amount * 0.10
```

○



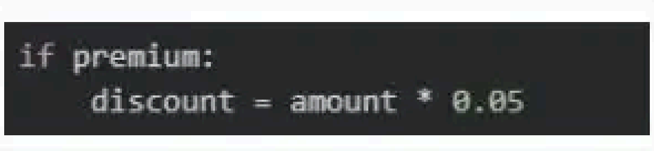
```python
if premium and amount >= 2000:
    discount = amount * 0.05
```

14) A **premium customer** makes a purchase of **₹2,000** exactly. *1 point*
Which discount amount is applied?

○ ₹100
● ₹200
○ ₹300
○ ₹0

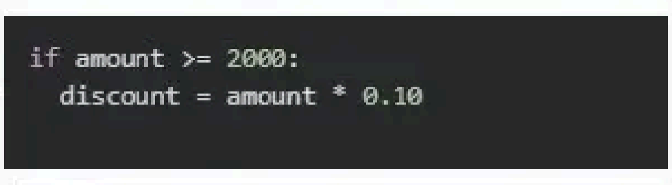
15) A **premium customer** makes a purchase of **₹1,800.** *1 point*
Which of the following code fragments correctly produces the discount applied by the Smart Discount Calculator?

○



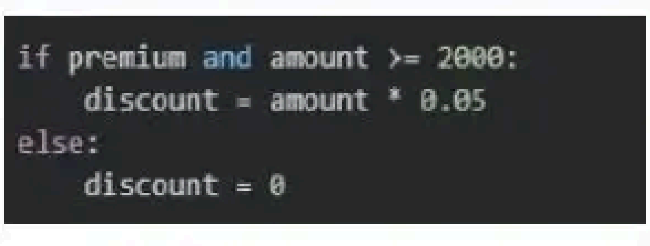
```python
if premium:
    discount = amount * 0.05
```

○



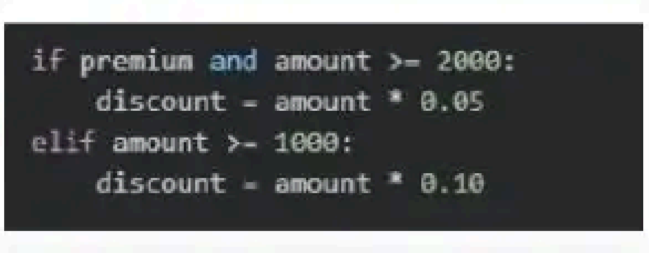
```python
if amount >= 2000:
  discount = amount * 0.10
```

○



```python
if premium and amount >= 2000:
    discount = amount * 0.05
else:
    discount = 0
```

●



```python
if premium and amount >= 2000:
    discount = amount * 0.05
elif amount >= 1000:
    discount = amount * 0.10
```

You may submit any number of times before the due date. The final submission will be considered for grading.

Submit Answers